



SIMATS ENGINEERING

CSA3703 – SOFTWARE TESTING

ASSIGNMENT 3

Analysis, Comparison and Justification of Multi-Level Testing Strategies for the RailEase Railway Ticket Reservation System

Submission detail	Information
Student Name	Srikesh K
Register Number	192321040
Department / Section	Information Technology
Course Code and Name	CSA3703 – Software Testing
Assignment Nature	Individual
Duration	1 week
Total Marks	50
Date of Submission	4-11-26
Faculty Name	Dr Gnanajeyaraman Rajaram

DECLARATION AND REPORT SCOPE

This report addresses the ten tasks in the supplied Software Testing Assignment 3 brief. It uses the RailEase scenario, the stated six-module call structure, fare and refund rules, quality targets, team capacity, tools, test assets, and the five-week delivery window.

Important evidence note: this document provides reproducible test designs, prototype code, example calculations, decision tables and illustrative planning estimates. It does not claim that tests were executed against a live RailEase system. Any measured execution time, defect count, coverage percentage, response-time percentile or volunteer feedback must be replaced with actual results from the student's own execution before submission.

EXECUTIVE SUMMARY

RailEase is a high-risk transaction system because a booking must reserve one berth only, payment and ticket status must agree, cancellation must apply the correct time slab, and search must meet the response-time target. The trial-week failures show that testing must be planned across levels rather than relying on user-interface testing near release.

The recommended baseline is a balanced testing pyramid: plan-driven, specification-led unit tests; mock-based tests for precise interaction checks supplemented by gateway virtualisation for network behaviour; incremental and thread-based integration; scenario-based integration planning; a system suite that combines functional, performance, recovery and security checks; impact-based regression with a prioritised smoke suite; and staged alpha, beta and formal acceptance.

The changed requirements in Task 10 make the original assumptions obsolete. With mobile, UPI, wallet, a new microservice, 15,000 simultaneous users, fortnightly releases and two testers, the strategy should be redesigned toward an automated multi-level CI pipeline, consumer-driven contract testing, repeatable load testing and staged rollout. AI-generated tests can accelerate drafting but must be reviewed against the stated business rules and retain traceable execution evidence.

TABLE OF CONTENTS

• Scenario and assumptions

• Task 1 – Unit test planning

• Task 2 – Unit test design styles

• Task 3 – Test harness construction

• Task 4 – Integration strategies

• Task 5 – Integration test planning

• Task 6 – System test focus areas

• Task 7 – Regression test selection

• Task 8 – Alpha, beta and acceptance

• Task 9 – Complete multi-level strategy

• Task 10 – Strategy adaptation

• Consolidated design decisions

• Implementation appendix and evidence templates

SCENARIO, GIVEN RULES AND ASSUMPTIONS

Area	Given requirement / rule	Testing implication
Modules	S Search; B Booking; P Payment; T Ticket; R Cancellation/Refund; N Notification	Test module behaviour and all six interfaces.
Call graph	S→B, B→P, P→T, T→N, R→P, R→N	Validate payloads, status, errors and order.
Fare	Sleeper ₹450; AC 3-tier ₹1,200; AC 2-tier ₹1,800	Test class-based price selection.
Tatkal	Adds 30%	Test normal/Tatkal combinations and rounding.
Senior concession	Age 60+ gets 40% concession on base fare	Age 59/60/61 boundaries; confirm order with Tatkal.
Passenger count	1–6 per ticket	Test 0, 1, 6, 7 and multiplication.
Booking horizon	120 days ahead	Test 119/120/121-day boundary.
Waitlist	Limit 50 per class	Test 49/50/51 and confirmation transitions.
GST	5% on AC only; none on Sleeper	Check tax class and calculation order.
Refund	≥48h: ₹60/passenger; 12–<48h: 25%; 4–<12h: 50%; <4h/no-show: no refund	Test exact boundaries and per-passenger calculation.
Quality targets	≤3 s at 1,500 concurrent users; 99.5% availability; no double berth; payment/ticket status agree	Use load, resilience, concurrency and consistency assertions.
Team/window	5 weeks; 3 testers, 4 developers, 1 clerk representative	Automate repetitive checks and reserve UAT time.

Assumptions to validate: currency is INR; fares are per passenger; the concession applies to the base fare as stated and the order of Tatkal, concession and GST must be confirmed by the commercial officer; refunds cannot be negative; timestamps use a single agreed time zone; a payment callback may be repeated; and all prototype measurements are illustrative until actually executed.

TASK 1 – UNIT TEST PLANNING

Marks: 4 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

Unit testing verifies the smallest testable functions before they are connected to other modules. For RailEase, ticket_fare(cls, passengers, age, tatkal) and refund_amount(hours_before, fare) are financially sensitive units. A wrong result can create overcharging, under-refunding or customer disputes.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Ad hoc developer tests	Quick to start; relies on individual judgement; coverage and expected outcomes vary; difficult to audit.	Useful for exploration but unsuitable as the sole strategy for fare/refund rules.
B. Plan-driven unit testing	Defines scope, inputs, techniques, responsibilities, tools, entry/exit criteria and evidence before execution.	Strong traceability and repeatability; can be slower to establish but reduces omissions.
C. Behaviour-driven unit testing	Uses Given–When–Then examples agreed with a booking-clerk representative and turns them into executable tests.	Clarifies business language and expected outcomes; does not alone guarantee structural coverage.

Application to RailEase

Unit / feature	Key tests	Entry criteria	Exit criteria
ticket_fare(cls, passengers, age, tatkal)	Class fare; 1/6/7 passengers; age 59/60/61; Tatkal off/on; AC GST vs Sleeper; rounding.	Requirement rules reviewed; test framework available; seeded-fault version identified.	All critical cases pass; 100% statement and branch coverage for these small functions or documented exception; no unexplained failures.

refund_amount(hours_before, fare)	49/48/47.99, 12.01/12/11.99, 4.01/4/3.99; no-show; negative values; multiple passengers.	Refund slab and per-passenger rule confirmed by commercial owner.	All slab boundaries tested; no known critical unit defect; expected values independently reviewed.

```
def refund_amount(hours_before, fare):
    if hours_before is None or hours_before < 0 or fare < 0:
        raise ValueError("invalid input")
    if hours_before >= 48:
        return max(0, fare - 60)
    if hours_before >= 12:
        return fare * 0.75
    if hours_before >= 4:
        return fare * 0.50
    return 0.0

# Important: confirm whether fare is per passenger or total ticket fare.
# Test exact boundaries and independently calculate expected values.
```

Recommendation

Adopt plan-driven testing with BDD examples for business rules. The plan gives consistent coverage and exit gates, while BDD examples ensure the commercial meaning is agreed. Ad hoc tests remain useful for exploratory checks, not as the control process.

Required evidence and cautions

Unit test plan, four seeded faults, fault-detection/coverage/effort table, AI-generated test review and a measurable exit criterion.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 2 – UNIT TEST DESIGN STYLES

Marks: 4 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

The refund function has four policy regions and critical boundary values at 48, 12 and 4 hours. A useful suite must test both business specification and implementation structure, while allowing new examples to be added without duplicating code.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Specification-based	Equivalence classes and boundary values derived from refund policy; black-box expected results.	Best at finding slab and invalid-input mistakes; readable as business requirements.
B. Structure-based	Cases selected to cover statements, branches and conditions in the implementation.	Finds unexecuted paths and conditional mistakes; can miss an incorrect business rule if expected values copy the code.
C. Parameterised data-driven	One test function reads rows of input and expected output.	Efficient maintenance and broad data coverage; failure messages need descriptive case IDs.

Application to RailEase

Case ID	Hours before	Fare (₹)	Expected refund (₹)	Purpose
RF-01	72	450	390	Above 48-hour slab

RF-02	48	450	390	Exact 48-hour boundary
RF-03	24	450	337.5	25% deduction
RF-04	12	450	337.5	Exact 12-hour boundary
RF-05	8	450	225	50% deduction
RF-06	4	450	225	Exact 4-hour boundary
RF-07	3.99	450	0	Less than 4 hours
RF-08	-1	450	Reject	Invalid negative time
RF-09	24	0	0	Zero fare
RF-10	None	450	0	No-show only if defined as no refund; model separately

Maintenance comparison: a specification-based suite changes the boundary data and any affected expectation; a structure-based suite may require updates to branch-focused cases; a parameterised suite generally changes one or more data rows, plus any shared expected-value helper. Count actual changed lines in the chosen implementation rather than presenting an estimate as a measured fact.

Recommendation

Use a parameterised table as the implementation format, populated from specification-based boundary analysis, and add structural coverage as a separate adequacy check. This combination is more reliable than treating the three styles as mutually exclusive.

Required evidence and cautions

Three suites with at least eight cases each, seeded faults, fault/coverage/maintenance comparison, LLM table audit and line-change estimate for moving the 12-hour boundary to 10 hours.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 3 – TEST HARNESS CONSTRUCTION

Marks: 4 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

Booking and cancellation depend on the external payment gateway. A harness allows these modules to be tested before the real service is available and enables controlled reproduction of success, decline, timeout and duplicate callback behaviour.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Hand-written drivers and stubs	Small custom functions return preselected outcomes; inexpensive and easy to understand.	Fast but may oversimplify real payloads, timing and duplicate delivery.
B. Mock-framework doubles	Mocks and spies verify arguments, call counts, order and expected interactions.	Excellent for deterministic unit/integration checks; not a substitute for network realism.
C. Gateway service virtualisation	A separate service replays realistic responses, delays, timeouts and error codes over the network interface.	More realistic and reusable; requires setup and maintenance of response scenarios.

Application to RailEase

Scenario	Hand stub	Mock framework	Virtual service
Gateway success	Fixed success response	Return success and verify amount/reference	HTTP success response over network

Decline	Return decline	Assert decline branch and no ticket call	Replay gateway decline code
Timeout	Usually hard-coded result	Raise timeout exception; timing limited	Configured delay/timeout
Duplicate callback	Must be manually modelled	Verify idempotency handler calls	Replay duplicate callback over interface
Partial failure: paid, ticket failed	Simulate T failure directly	Assert payment succeeded but ticket retry/reconciliation occurs	Exercise gateway and downstream failure sequence
Main limitation	Stub realism	Mock interaction assumptions	Setup/maintenance cost

Recommendation

Use mock-framework doubles for fast isolated checks and virtualisation for the highest-risk gateway behaviours. Keep a small hand-written stub only for simple deterministic cases. The evidence must show the limits of each harness rather than assuming a mock equals the production gateway.

Required evidence and cautions

Three harness designs, six or more cases per harness, effort/runtime/fault table and explicit duplicate-callback and partial-failure cases.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 4 – INTEGRATION TESTING STRATEGIES

Marks: 5 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

Integration testing checks whether modules exchange correct data and handle failure consistently. RailEase has a clear call graph but also a high-risk payment-to-ticket dependency and refund calls to payment and notification.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Big-bang	All six modules are connected and tested together through the UI.	End-to-end realism, but failures are difficult to localise and dependencies delay first useful results.
B. Incremental top-down/bottom-up	Modules are integrated in steps, with stubs or drivers for unfinished neighbours.	Faults are localised earlier; temporary doubles must be managed and replaced.
C. Thread-based	One user-visible thread is integrated end-to-end at a time, such as search→book→pay→ticket→notify.	Delivers early usable journeys and exposes cross-module data issues along a business path.

Application to RailEase

RailEase module dependency graph



Cancellation R also calls Payment P and Notification N. The P→T dependency must enforce that a paid transaction is reconciled if ticket generation fails; R→P must be idempotent to prevent duplicate refunds.

Strategy	Likely order	Doubles needed	Fault localisation
Big-bang	S+B+P+T+N+R at once	Few temporary doubles; many real dependencies at once	Low
Top-down	S→B→P→T→N, then R paths	Stubs for lower dependencies not ready	High if interfaces are checked at each step
Bottom-up	P/T/N components first, then B/S; R integration	Drivers for higher-level callers	High but end-to-end feedback arrives later
Thread-based	Search→book→pay→ticket→notify; then cancel→refund→notify	Targeted stubs around unfinished edges	High along business journey

Recommendation

Choose thread-based integration supported by incremental interface checks. Begin with search-to-book and payment-to-ticket, then add notification and cancellation/refund. Exercise R→P and R→N carefully to avoid duplicate refunds and duplicate notifications. Do not defer the external payment boundary until the end.

Required evidence and cautions

Dependency diagram, integration orders, stub/driver inventory, five seeded interface faults and time-to-localise comparison.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 5 – INTEGRATION TEST PLANNING

Marks: 5 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

An integration plan must make interfaces, end-to-end scenarios, build readiness and acceptance criteria visible to developers, testers and the commercial officer. A plan that only follows the calendar can leave a business-critical interface uncovered.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Interface-based plan	One section per S→B, B→P, P→T, T→N, R→P and R→N interface.	Strong payload/protocol/error coverage; can understate full user journey behaviour.
B. Scenario-based plan	Plans around book/pay/ticket and cancel/refund/notify journeys.	Strong business traceability and end-to-end visibility; can hide an interface not exercised by chosen scenarios.
C. Build-schedule plan	Plans by weekly build with entry and exit criteria.	Clear sequencing and ownership; schedule changes can leave gaps unless coverage is mapped to interfaces.

Application to RailEase

B→P test	Input / event	Expected result
IP-01 Amount/currency	₹450 and INR	Gateway receives exact rupee amount; no paise/rupee conversion error.
IP-02 Timeout	Gateway delays beyond timeout	Booking is pending/failed safely; retry is idempotent.
IP-03 Duplicate request	Same idempotency key twice	One charge only.
IP-04 Decline	Issuer decline response	No ticket is issued; user sees actionable status.

IP-05 Partial failure	Payment succeeds, T fails	Reconciliation/retry or compensating workflow; status remains consistent.
IP-06 Retry	Transient error then success	One final charge and one ticket.
IP-07 Passenger count	6 passengers; malformed 7	Valid count passed correctly; invalid count rejected before payment.
IP-08 Late callback	Callback arrives after session expiry	Callback reconciled securely; no duplicate booking/charge.

Recommendation

Approve a hybrid plan: organise work by build, maintain an explicit interface checklist, and gate each build with scenario outcomes. B→P requires amount/currency, timeout, duplicate payment, partial failure, retry, passenger count and callback-after-expiry tests.

Required evidence and cautions

Three plan variants, scored checklist, eight B→P tests, late Notification module contingency, entry/exit criteria and LLM plan-review checklist.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 6 – SYSTEM TEST FOCUS AREAS

Marks: 5 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

System testing validates the complete deployed solution against functional, performance, security, recovery and compatibility requirements. The trial week points to three distinct risk families: double berth allocation, payment without ticket and slow search under concurrency.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Functional/usability	End-to-end booking, cancellation, refund, ticket display and clerk/traveller usability.	Strong business correctness; may miss high-load and recovery failures.
B. Performance/load/stress	JMeter search-and-book flows at 400, 1,500 and 3,000 concurrent users; measure average and p90.	Directly addresses the 3-second target and saturation; does not prove every business rule.
C. Recovery/security/compatibility	Restart/failure recovery, session and injection tests, browsers and mobile devices.	Finds resilience and security risks; cannot replace core functional and performance testing.

Application to RailEase

Focus area	Example cases	Main risk covered	Key measure
Functional/usability	Search, book, cancel, refund, ticket, notification; clerk task observation	Double berth, incorrect refund, payment/ticket mismatch	Pass rate, severity, task completion time
Performance/load	400, 1,500, 3,000 concurrent users; ramp and soak tests	Slow search and saturation	Average, p90, errors/sec, throughput
Recovery/security/compatibility	Gateway timeout, restart, session expiry, injection, browsers/mobile	Data loss, access abuse, inconsistent state	Recovery time, vulnerability severity, compatibility failures

```
# JMeter / equivalent run plan (conceptual)
# 1. Prepare a production-like data set and a sandbox gateway.
# 2. Ramp to 400, 1500, then 3000 virtual users.
# 3. Execute search -> select train -> book -> pay -> ticket.
# 4. Record average and p90 response time, errors and throughput.
# 5. Repeat after fixes; preserve the raw JTL/CSV and test configuration.
# Never invent response-time results; attach the output from the actual run.
```

Recommendation

Use a risk-weighted blend rather than a single exclusive focus: functional integrity first, performance testing repeatedly as builds stabilise, then recovery/security/compatibility in parallel. Report real load metrics only after running the script in a defined environment.

Required evidence and cautions

At least eight cases per focus area, JMeter plan, load chart template, defect/severity/effort log and staging-production limitation analysis.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 7 – REGRESSION TEST SELECTION

Marks: 5 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

Weekly changes to fare and refund rules create regression risk. The full 650-case manual suite takes three testers four days, so retesting everything after every change is expensive; however, selecting too narrowly can miss defects outside the mapped area.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Retest-all	Runs every existing case after each change.	Highest broad reuse and low selection risk, but high time and opportunity cost.
B. Impact-based selection	Runs tests linked to changed modules and callers through traceability.	Efficient when traceability is current; can miss tests when mappings are stale or dependencies are hidden.
C. Prioritised suite	Orders tests by history, changed-code coverage and business criticality.	Finds important defects early within a time budget; may defer lower-priority faults.

Application to RailEase

Metric	Formula	Interpretation
Fault detection %	(Seeded faults detected ÷ seeded faults) × 100	Use same seeded-fault set for fair comparison.
Time saved	Retest-all time − selected-suite time	Compare wall-clock and tester effort separately.
APFD	$1 - (\Sigma\ TF_i)/(n \times m) + 1/(2n)$	n = tests, m = faults, TF_i = first test position detecting fault i; use a consistent definition.
Stale traceability impact	Fault linked tests omitted ÷ all fault-relevant tests	Document a deliberate stale mapping scenario to expose risk.

Recommendation

Use a prioritised smoke suite first, then impact-based regression, and schedule periodic full-suite runs. Keep high-risk invariants—no duplicate berth, payment-ticket agreement, refund boundaries and booking limits—in the smoke suite even when the change appears unrelated.

Required evidence and cautions

Representative 60-case suite, 3 seeded regression faults, selected counts/runtime, detection %, time saved, APFD calculation and stale-traceability analysis.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 8 – ALPHA, BETA AND ACCEPTANCE TESTING

Marks: 4 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

User-level testing checks whether RailEase supports real working practices and meets the commercial officer's agreed acceptance conditions. These methods answer different questions and should not be treated as interchangeable.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Alpha	Seven internal users—four clerks and three agents—test under observation in a controlled environment.	Facilitator can reproduce and clarify issues; users may be biased or overly guided.
B. Beta	Fifty travellers use staging from their own devices and provide structured feedback.	Reveals device and usability diversity; sample is not statistically representative of 150,000 daily travellers.
C. Formal acceptance	Commercial officer signs measurable business, operational and legal criteria.	Provides an accountable release decision; only useful when criteria and unresolved-defect rules are clear.

Application to RailEase

Acceptance ID	Criterion	Evidence / threshold
AT-01	Booking completion	At least 95% of representative bookings complete in under 3 minutes.
AT-02	Berth uniqueness	Zero duplicate berth assignments in concurrency test.
AT-03	Payment-ticket consistency	100% of successful payments reconcile to a ticket or controlled exception.
AT-04	Refund slab	All approved boundary examples return expected amounts.
AT-05	Fare correctness	Fare agrees with approved fare table for all selected classes and concessions.
AT-06	Booking limits	Reject 0 and 7 passengers; accept 1–6.
AT-07	Waitlist limit	No class waitlist exceeds 50.
AT-08	Performance	Response time $\leq$3 seconds at 1,500 concurrent users under defined workload.
AT-09	Availability	Evidence supports 99.5% availability over the agreed observation period.
AT-10	Notification	Correct e-mail/SMS sent once for each eligible event; failures logged.

Recommendation

Run alpha after core workflows stabilise, beta after critical alpha defects are fixed, then formal acceptance against pre-agreed measurable criteria. No critical berth, payment, refund or data-security defect should be accepted merely because beta users did not report it.

Required evidence and cautions

Alpha plan, beta feedback form, ten acceptance cases, issue classification, 5-week schedule and limitations of the 50-user sample.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 9 – INTEGRATED MULTI-LEVEL TEST STRATEGY

Marks: 6 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

The strategy combines unit, integration, system, regression and user-level tests. A bottom-heavy strategy may miss realistic load and user workflow risks; a top-heavy strategy discovers basic logic defects late; a balanced pyramid attempts to find defects at the cheapest reliable level.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Bottom-heavy	Strong unit/integration with minimal system and acceptance testing.	Efficient for logic but weak for peak load, environment, usability and external gateway behaviour.
B. Top-heavy	Little unit testing; most effort on system/UAT.	Realistic but slow and expensive to diagnose; repeatability and regression become poor.
C. Balanced pyramid	Many automated units, moderate integration, focused system/performance and staged acceptance.	Balances cost, early fault isolation, end-to-end confidence and regression speed.

Application to RailEase

Level	Entry criteria	Exit criteria	Suggested effort share
Unit	Reviewed requirements and buildable modules	Critical tests pass; coverage and seeded-fault results reviewed	25%
Integration	Unit exit met; interface contracts available	All critical interfaces and journeys pass	20%
System	Integrated release candidate in staging	Critical functional/security/performance risks within limits	25%
Regression	Change impact and traceability available	Smoke plus selected regression pass; no unexplained failures	15%
Alpha/beta/UAT	Stable staging build; test data and users prepared	Acceptance criteria met or approved residual-risk decision	15%

Recommendation

Select the balanced pyramid. Record the point each seeded fault is detected, person-days and rework rather than relying on theoretical claims. Compare the results against expectations and explain any disagreement, particularly when historical anonymised data fails to reproduce festival peaks.

Required evidence and cautions

Ten-fault prototype, level-by-level results, complete strategy block diagram, design decision table, comparison across airline/ATM/school systems and AI-assisted option.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

TASK 10 – RE-ANALYSE AND ADAPT

Marks: 8 | Primary objective: compare all three prescribed techniques, analyse evidence, and justify a defensible choice.

The new mobile app, UPI and wallet, seat-recommendation microservice, 15,000-user Tatkal peak, fortnightly releases, two testers and mandatory critical-test evidence materially change the original plan. The five-week, three-tester assumptions cannot simply be reused.

Comparison of the three approaches

Technique	Analysis: strengths and limitations	Decision implication
A. Retain and prioritise	Keep the old strategy and shorten beta; run high-risk tests only.	Low transition cost but weak scalability and traceable evidence; likely to overload two testers.
B. Automated multi-level pipeline	Automate unit, integration and regression in CI; retain manual system and acceptance checks.	Improves repeatability and evidence, but pipeline quality and ownership must be established.
C. Contract tests + staged rollout	Consumer-driven contracts, automated load checks each release and canary/staged rollout.	Best suited to microservice boundaries and frequent releases; requires observability, rollback and contract governance.

Application to RailEase

Changed assumption	Old position	Required adaptation
Architecture	Six modules	Add mobile clients, UPI/wallet adapters and seat-recommendation service contracts.
Traffic	1,500 concurrent target	Test Tatkal ramp to 15,000; model arrival spikes and queue saturation.
Release cycle	Five-week pre-release window	Fortnightly CI gates and risk-based staged release.
Team	Three testers	Two testers; automate repeatable suites and assign developer ownership.
Evidence	Manual traceability sheet	Machine-readable run records linked to build, requirement, test, result and timestamp.
Rollout	Long beta phase	Canary/staged rollout with health thresholds, monitoring and rollback.

Recommendation

Recommend REDESIGN using a CI-based automated multi-level pipeline plus consumer-driven contracts, repeatable load tests and staged rollout. This is a targeted redesign, not the removal of manual acceptance: human review remains necessary for commercial sign-off, usability and release risk.

Required evidence and cautions

Compare A/B/C with actual prototype results, recalculated person-hours, 15,000-user capacity plan, critical-test evidence, AI selection controls and a condition that would change the decision.

Evidence integrity: do not label illustrative tables as observed measurements. For each executed experiment, record the software revision, environment, dataset, test command, start/end time, expected result, actual result, defect ID and reviewer.

ANNEXURE A3.01 – Fare class equivalence partitions

Purpose. This annexure gives a focused, practical test design for fare class equivalence partitions in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

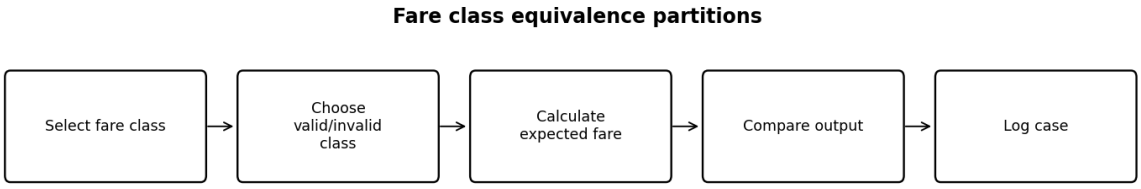


Figure A3.01. Fare class equivalence partitions – illustrative verification flow.

Representative automated check

```
assert ticket_fare("SLEEPER", 1, 59, False) == 450
assert ticket_fare("AC3", 1, 60, False) == expected_ac3_senior
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.02 – Passenger-count boundaries

Purpose. This annexure gives a focused, practical test design for passenger-count boundaries in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

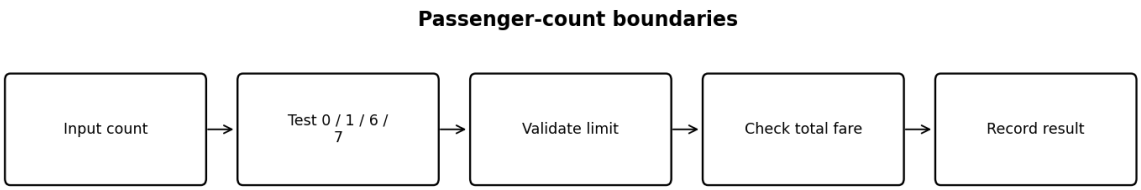


Figure A3.02. Passenger-count boundaries – illustrative verification flow.

Representative automated check

```
for n in [0, 1, 6, 7]:
    result = validate_passengers(n)
    assert result == (1 <= n <= 6)
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.03 – Senior concession boundaries

Purpose. This annexure gives a focused, practical test design for senior concession boundaries in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

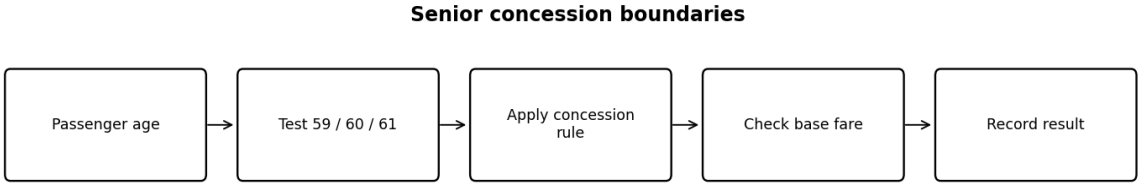


Figure A3.03. Senior concession boundaries – illustrative verification flow.

Representative automated check

```
assert senior_discount_applies(59) is False
assert senior_discount_applies(60) is True
assert senior_discount_applies(61) is True
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.

Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.04 – Tatkal fare adjustment

Purpose. This annexure gives a focused, practical test design for tatkal fare adjustment in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

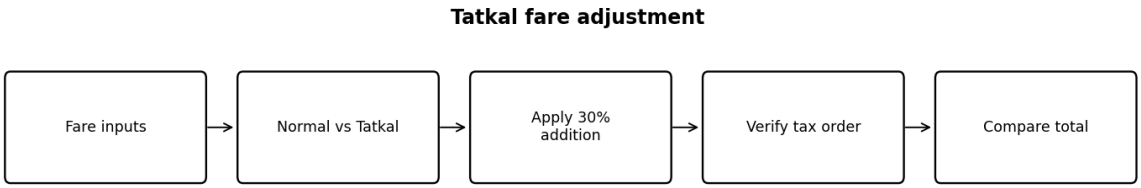


Figure A3.04. Tatkal fare adjustment – illustrative verification flow.

Representative automated check

```
normal = ticket_fare("AC3", 1, 40, False)
tatkal = ticket_fare("AC3", 1, 40, True)
assert tatkal >= normal
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.05 – GST by class

Purpose. This annexure gives a focused, practical test design for gst by class in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

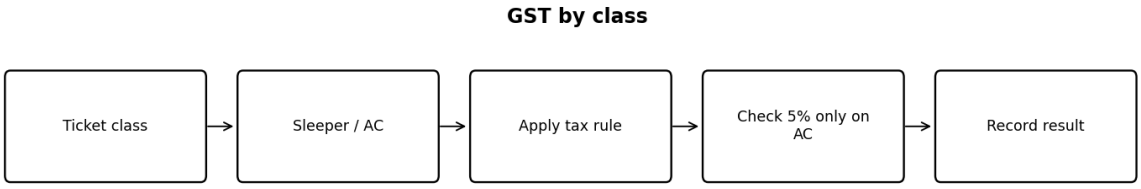


Figure A3.05. GST by class – illustrative verification flow.

Representative automated check

```
assert gst_for("SLEEPER", 450) == 0
assert gst_for("AC3", 1200) == 60
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.06 – Rounding and currency

Purpose. This annexure gives a focused, practical test design for rounding and currency in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

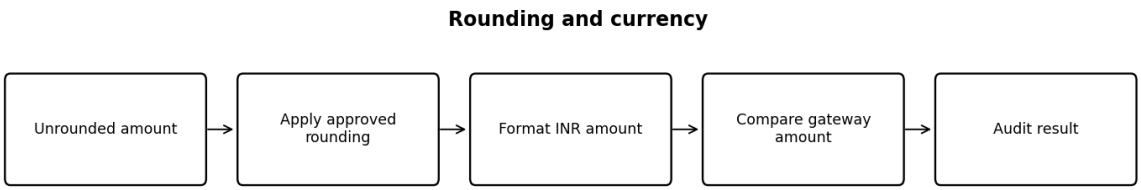


Figure A3.06. Rounding and currency – illustrative verification flow.

Representative automated check

```
assert format_inr(round(450.125, 2)) == "₹450.13"
# Use the approved rounding policy in the real implementation.
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.07 – Refund at 48 hours

Purpose. This annexure gives a focused, practical test design for refund at 48 hours in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

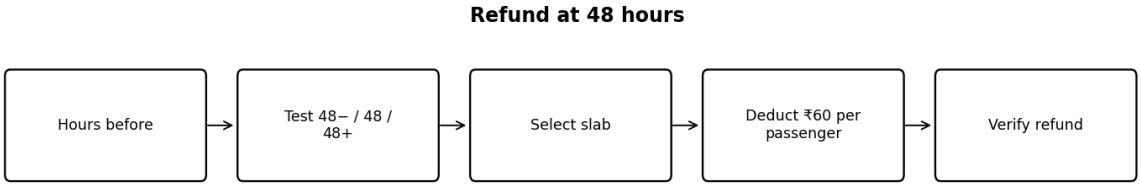


Figure A3.07. Refund at 48 hours – illustrative verification flow.

Representative automated check

```
assert refund_amount(48, 450) == 390
assert refund_amount(47.99, 450) == 337.50
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.08 – Refund at 12 hours

Purpose. This annexure gives a focused, practical test design for refund at 12 hours in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

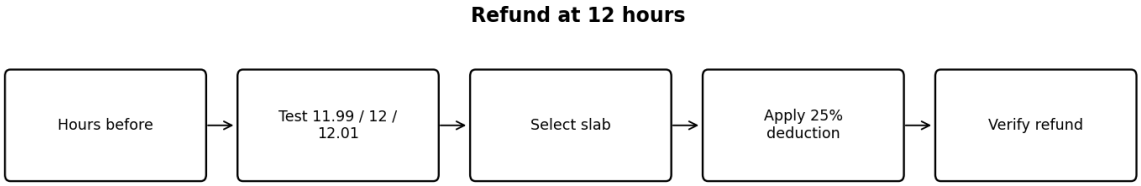


Figure A3.08. Refund at 12 hours – illustrative verification flow.

Representative automated check

```
assert refund_amount(12, 450) == 337.50
assert refund_amount(11.99, 450) == 225
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.09 – Refund at 4 hours

Purpose. This annexure gives a focused, practical test design for refund at 4 hours in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

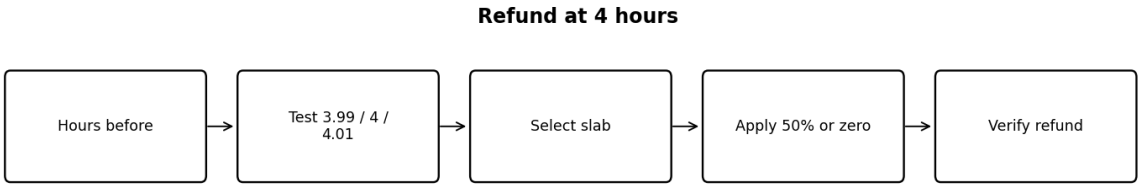


Figure A3.09. Refund at 4 hours – illustrative verification flow.

Representative automated check

```
assert refund_amount(4, 450) == 225
assert refund_amount(3.99, 450) == 0
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.10 – No-show handling

Purpose. This annexure gives a focused, practical test design for no-show handling in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

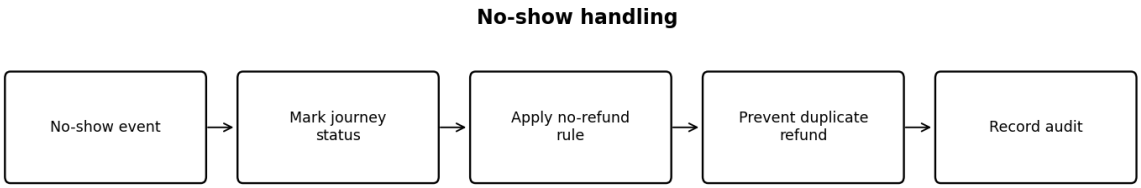


Figure A3.10. No-show handling – illustrative verification flow.

Representative automated check

```
assert refund_for_no_show(450) == 0
assert duplicate_refund_requests(transaction_id) == 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.11 – Waitlist capacity

Purpose. This annexure gives a focused, practical test design for waitlist capacity in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

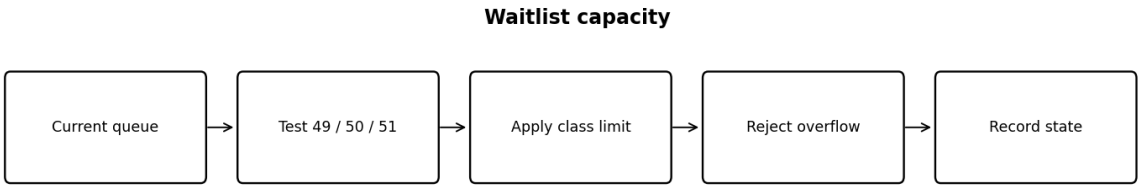


Figure A3.11. Waitlist capacity – illustrative verification flow.

Representative automated check

```
assert waitlist_count(49) <= 50
assert reject_waitlist_request(51) is True
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.12 – 120-day booking window

Purpose. This annexure gives a focused, practical test design for 120-day booking window in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

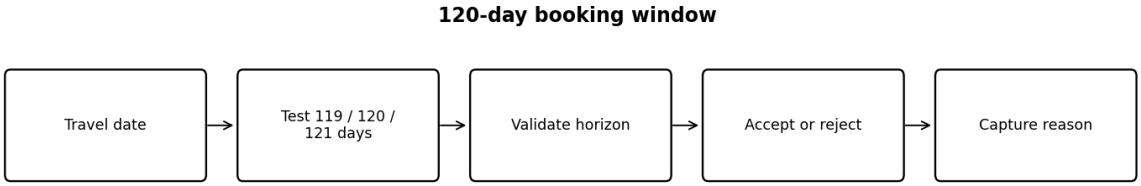


Figure A3.12. 120-day booking window – illustrative verification flow.

Representative automated check

```
assert can_book(days_ahead=120) is True
assert can_book(days_ahead=121) is False
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.13 – Double berth concurrency

Purpose. This annexure gives a focused, practical test design for double berth concurrency in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

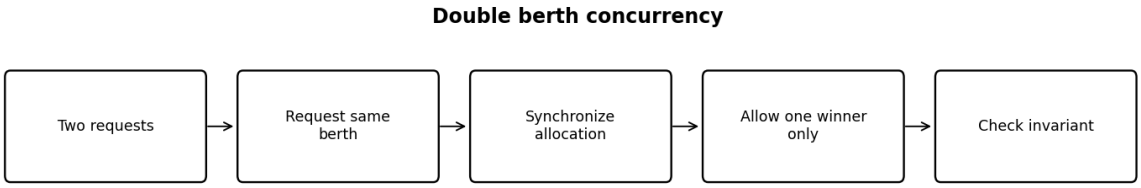


Figure A3.13. Double berth concurrency – illustrative verification flow.

Representative automated check

```
assert allocate_same_berth_concurrently(two_requests) == 1
# Exactly one request may own a berth.
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.14 – Payment success/ticket failure

Purpose. This annexure gives a focused, practical test design for payment success/ticket failure in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

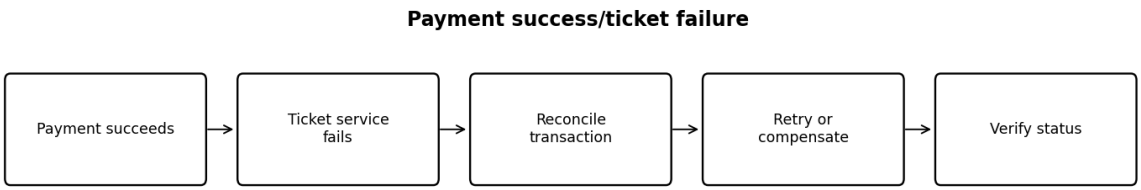


Figure A3.14. Payment success/ticket failure – illustrative verification flow.

Representative automated check

```
assert payment_status == "SUCCESS"
assert ticket_status in ["ISSUED", "RECONCILIATION_PENDING"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.15 – Duplicate payment callback

Purpose. This annexure gives a focused, practical test design for duplicate payment callback in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

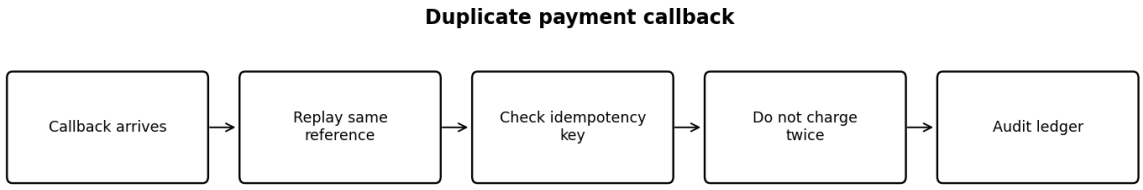


Figure A3.15. Duplicate payment callback – illustrative verification flow.

Representative automated check

```
process_callback(callback)
process_callback(callback)
assert ledger.charge_count(payment_ref) == 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.

Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.16 – Gateway decline

Purpose. This annexure gives a focused, practical test design for gateway decline in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

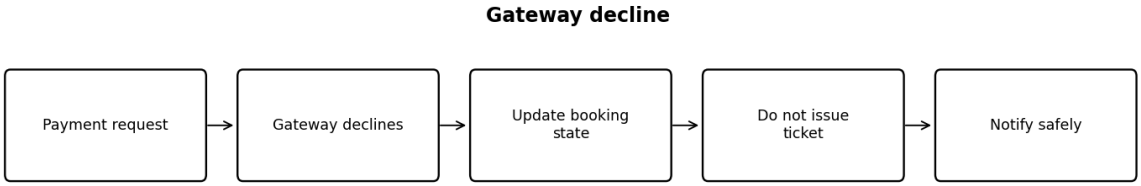


Figure A3.16. Gateway decline – illustrative verification flow.

Representative automated check

```
assert declined_payment.ticket_id is None
assert declined_payment.status == "PAYMENT_DECLINED"
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.17 – Gateway timeout

Purpose. This annexure gives a focused, practical test design for gateway timeout in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

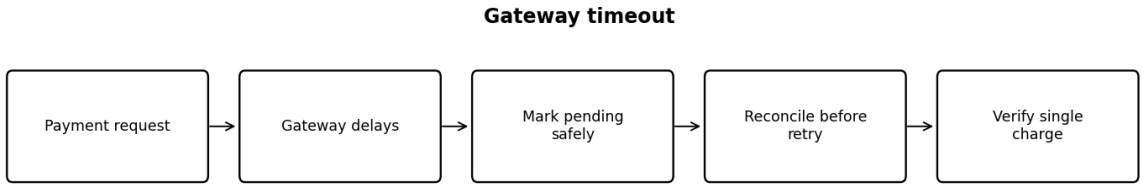


Figure A3.17. Gateway timeout – illustrative verification flow.

Representative automated check

```
assert retry_payment(timeout_ref) is idempotent
assert gateway_charge_count(timeout_ref) <= 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.18 – Callback after session expiry

Purpose. This annexure gives a focused, practical test design for callback after session expiry in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

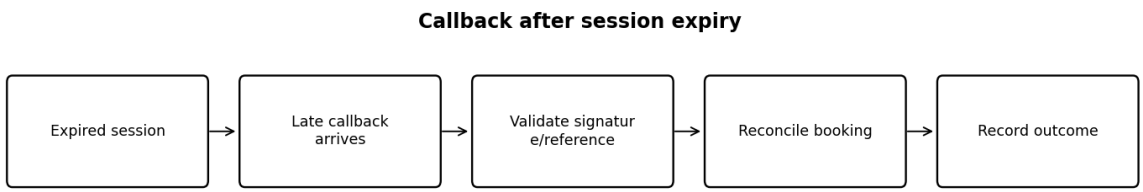


Figure A3.18. Callback after session expiry – illustrative verification flow.

Representative automated check

```
assert validate_callback_signature(callback)
assert reconcile_late_callback(callback).is_audited
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.19 – B-to-P interface payload

Purpose. This annexure gives a focused, practical test design for b-to-p interface payload in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

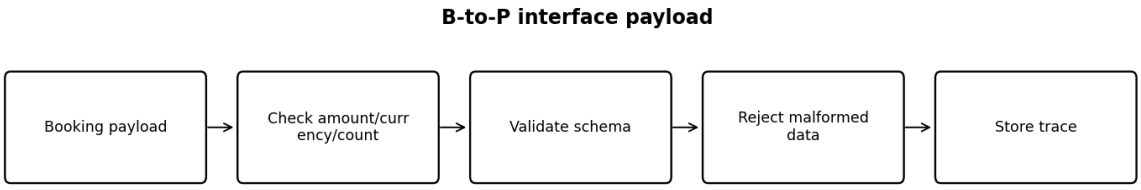


Figure A3.19. B-to-P interface payload – illustrative verification flow.

Representative automated check

```
assert payload["currency"] == "INR"
assert 1 <= payload["passenger_count"] <= 6
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.20 – P-to-T interface status

Purpose. This annexure gives a focused, practical test design for p-to-t interface status in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

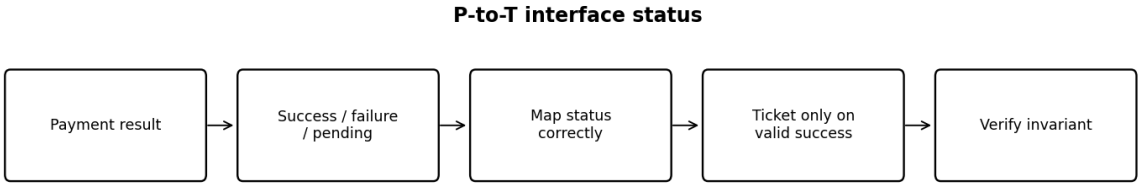


Figure A3.20. P-to-T interface status – illustrative verification flow.

Representative automated check

```
assert ticket_issued only_if payment_status == "SUCCESS"
assert status_pair_is_consistent(payment_status, ticket_status)
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.21 – T-to-N notification payload

Purpose. This annexure gives a focused, practical test design for t-to-n notification payload in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

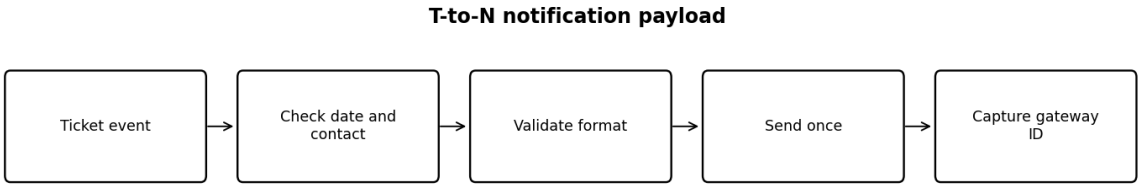


Figure A3.21. T-to-N notification payload – illustrative verification flow.

Representative automated check

```
assert notification.train_date_format == "YYYY-MM-DD"
assert notification.sent_count <= 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.22 – R-to-P refund request

Purpose. This annexure gives a focused, practical test design for r-to-p refund request in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

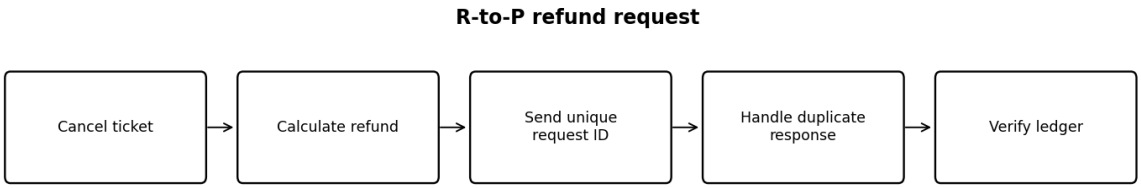


Figure A3.22. R-to-P refund request – illustrative verification flow.

Representative automated check

```
assert refund_request.idempotency_key != ""
assert refund_ledger.count(refund_request.idempotency_key) <= 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.23 – R-to-N cancellation notice

Purpose. This annexure gives a focused, practical test design for r-to-n cancellation notice in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

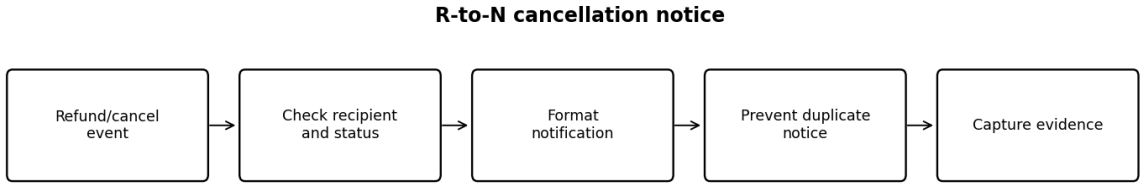


Figure A3.23. R-to-N cancellation notice – illustrative verification flow.

Representative automated check

```
assert cancellation_notice.booking_id == booking.id
assert notification_count(booking.id, "CANCELLED") <= 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.24 – Big-bang integration risk

Purpose. This annexure gives a focused, practical test design for big-bang integration risk in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

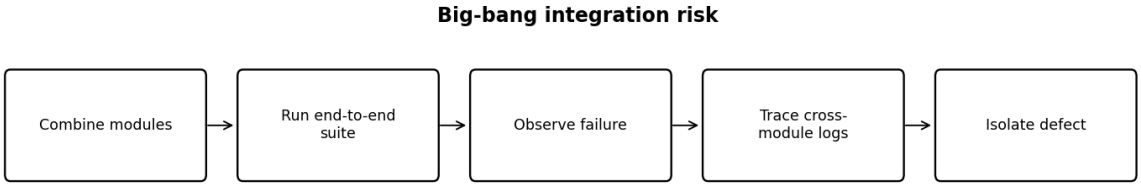


Figure A3.24. Big-bang integration risk – illustrative verification flow.

Representative automated check

```
for module in [S, B, P, T, N, R]:
    assert module.health_check() in ["OK", "TEST_STUB"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.25 – Top-down integration

Purpose. This annexure gives a focused, practical test design for top-down integration in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

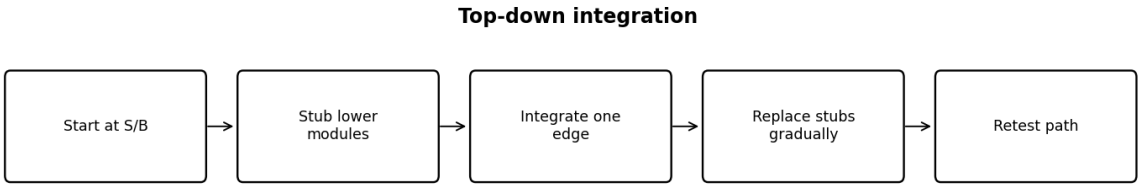


Figure A3.25. Top-down integration – illustrative verification flow.

Representative automated check

```
assert booking_stub.search_called_once()
assert payment_stub.received_correct_amount()
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.26 – Bottom-up integration

Purpose. This annexure gives a focused, practical test design for bottom-up integration in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

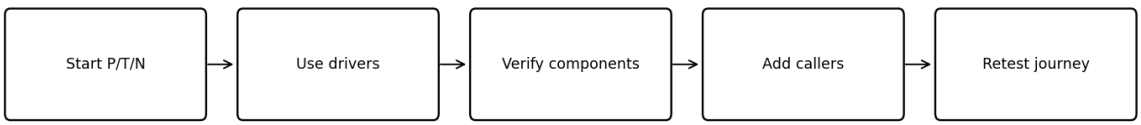


Figure A3.26. Bottom-up integration – illustrative verification flow.

Representative automated check

```
assert payment_component.approve(valid_request)
assert ticket_component.reject(invalid_payment_state)
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.27 – Thread-based integration

Purpose. This annexure gives a focused, practical test design for thread-based integration in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

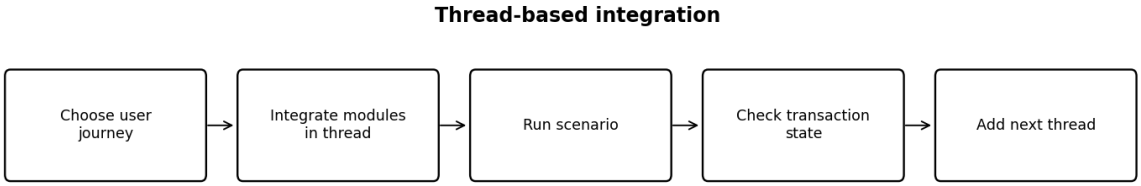


Figure A3.27. Thread-based integration – illustrative verification flow.

Representative automated check

```
assert scenario("search-book-pay-ticket").completed
assert scenario("cancel-refund-notify").completed
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.28 – Interface contract test

Purpose. This annexure gives a focused, practical test design for interface contract test in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

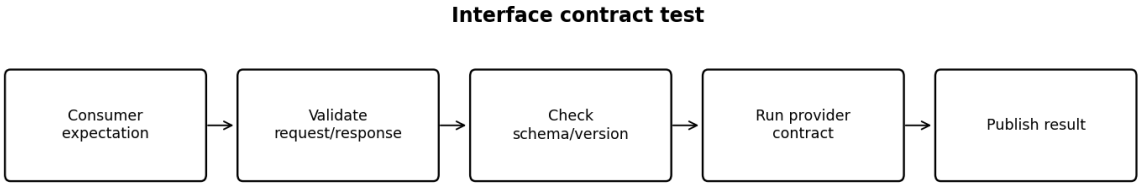


Figure A3.28. Interface contract test – illustrative verification flow.

Representative automated check

```
assert provider_response.matches_contract(expected_schema)
assert contract_version == supported_contract_version
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.29 – Late notification module

Purpose. This annexure gives a focused, practical test design for late notification module in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

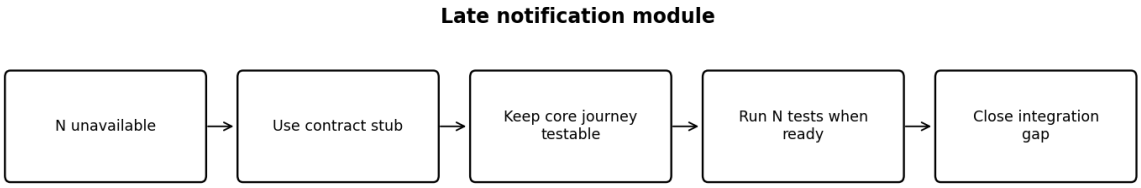


Figure A3.29. Late notification module – illustrative verification flow.

Representative automated check

```
assert core_booking_tests_can_run_without_N
assert notification_contract_suite_runs_when_N_ready
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.30 – Build entry gate

Purpose. This annexure gives a focused, practical test design for build entry gate in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

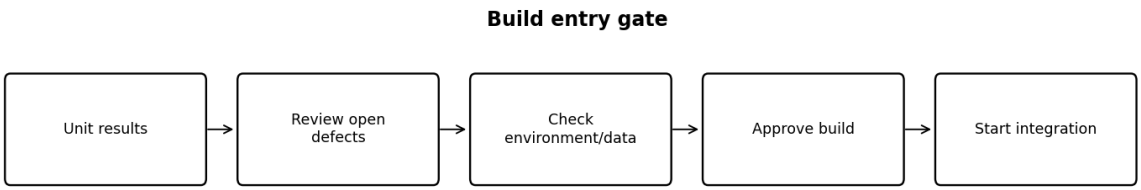


Figure A3.30. Build entry gate – illustrative verification flow.

Representative automated check

```
assert unit_gate_passed
assert critical_open_defects == 0
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.31 – Build exit gate

Purpose. This annexure gives a focused, practical test design for build exit gate in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

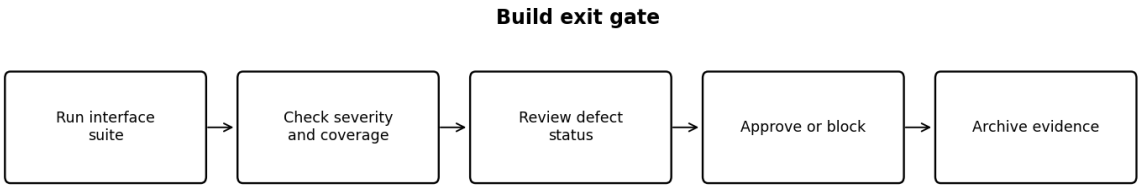


Figure A3.31. Build exit gate – illustrative verification flow.

Representative automated check

```
assert critical_interface_tests_passed
assert unresolved_blockers == 0
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.32 – Functional booking flow

Purpose. This annexure gives a focused, practical test design for functional booking flow in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

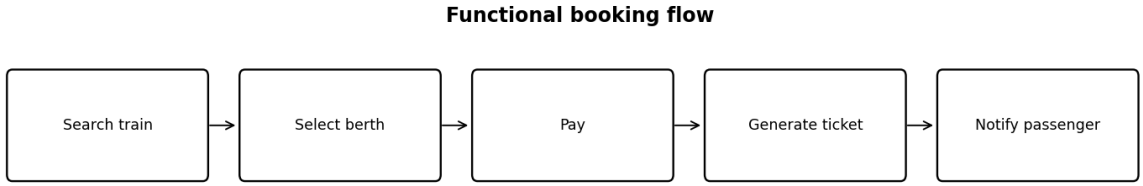


Figure A3.32. Functional booking flow – illustrative verification flow.

Representative automated check

```
assert journey_result.ticket_status == "ISSUED"
assert journey_result.payment_status == "SUCCESS"
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.33 – Cancellation end-to-end

Purpose. This annexure gives a focused, practical test design for cancellation end-to-end in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

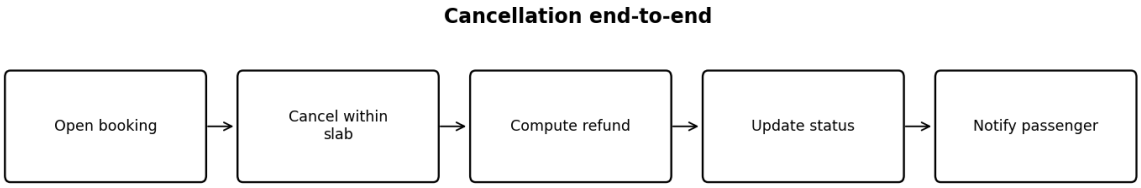


Figure A3.33. Cancellation end-to-end – illustrative verification flow.

Representative automated check

```
assert cancellation.refund_amount == expected_refund
assert cancellation.notification_status in ["SENT", "RETRY_PENDING"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.34 – Load test ramp-up

Purpose. This annexure gives a focused, practical test design for load test ramp-up in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

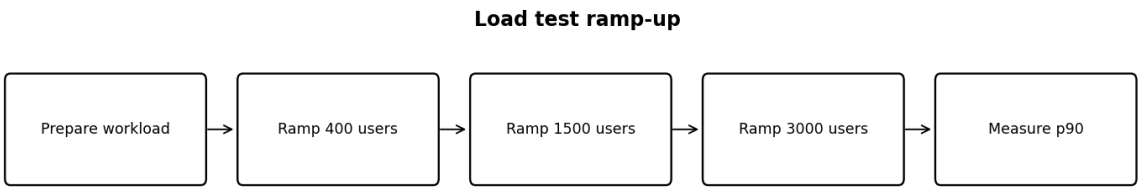


Figure A3.34. Load test ramp-up – illustrative verification flow.

Representative automated check

```
assert report.p90_ms <= 3000
assert report.error_rate <= agreed_error_threshold
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.35 – Load test at target

Purpose. This annexure gives a focused, practical test design for load test at target in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

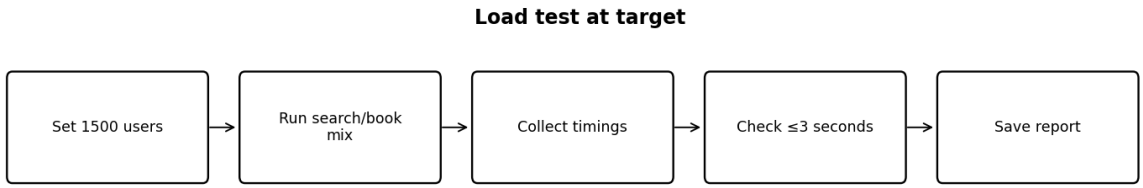


Figure A3.35. Load test at target – illustrative verification flow.

Representative automated check

```
assert run.users == 1500
assert report.search_p90_ms <= 3000
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.36 – Stress beyond target

Purpose. This annexure gives a focused, practical test design for stress beyond target in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

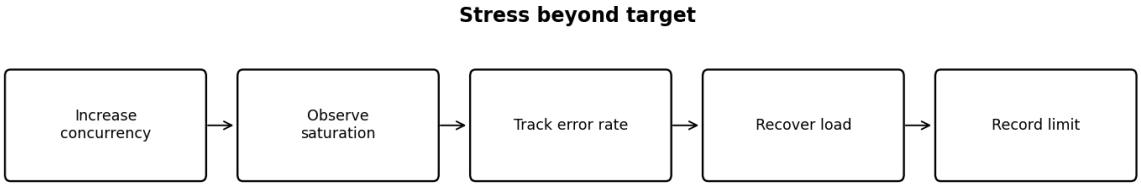


Figure A3.36. Stress beyond target – illustrative verification flow.

Representative automated check

```
assert report.max_sustainable_users >= target_capacity
assert report.recovery_completed is True
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.37 – Soak and endurance

Purpose. This annexure gives a focused, practical test design for soak and endurance in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

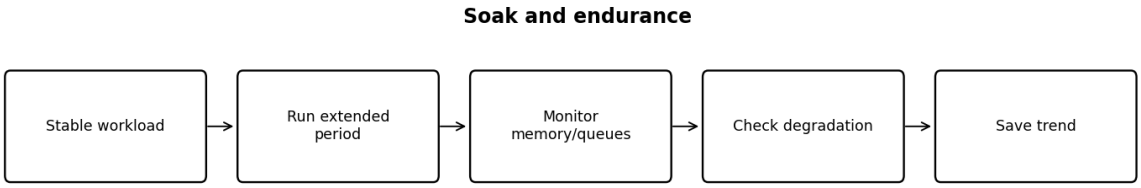


Figure A3.37. Soak and endurance – illustrative verification flow.

Representative automated check

```
assert degradation_slope <= agreed_limit
assert no_unbounded_queue_growth
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.38 – Availability evidence

Purpose. This annexure gives a focused, practical test design for availability evidence in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

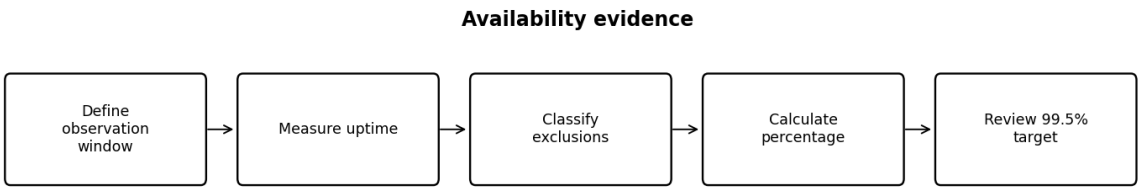


Figure A3.38. Availability evidence – illustrative verification flow.

Representative automated check

```
availability = uptime_seconds / observed_seconds * 100
assert availability >= 99.5
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.39 – Security session checks

Purpose. This annexure gives a focused, practical test design for security session checks in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.



Figure A3.39. Security session checks – illustrative verification flow.

Representative automated check

```
assert expired_session.response_status in [401, 403]
assert audit_log.contains("SESSION_EXPIRED")
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.40 – Injection validation

Purpose. This annexure gives a focused, practical test design for injection validation in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

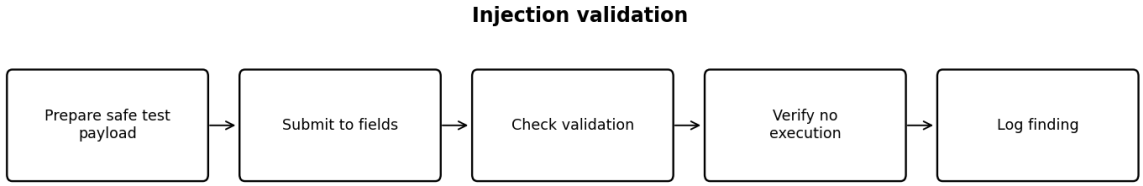


Figure A3.40. Injection validation – illustrative verification flow.

Representative automated check

```
assert injection_test.no_code_execution
assert injection_test.input_was_logged_safely
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.41 – Browser compatibility

Purpose. This annexure gives a focused, practical test design for browser compatibility in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

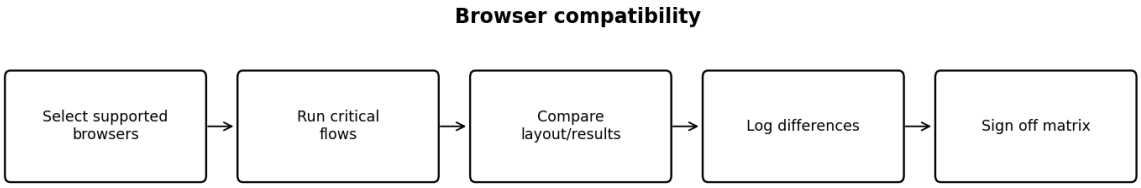


Figure A3.41. Browser compatibility – illustrative verification flow.

Representative automated check

```
assert critical_flow_results["supported_browser"] == "PASS"
assert screenshot_diff_within_tolerance
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.42 – Mobile compatibility

Purpose. This annexure gives a focused, practical test design for mobile compatibility in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

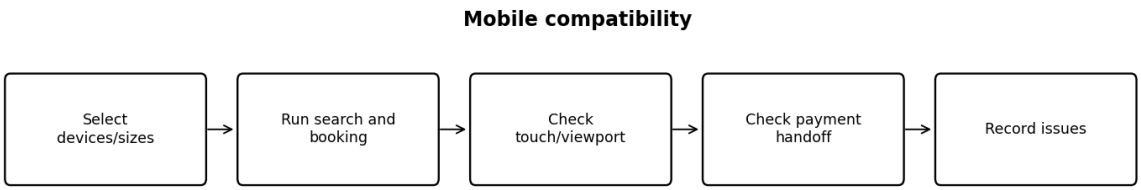


Figure A3.42. Mobile compatibility – illustrative verification flow.

Representative automated check

```
assert mobile_booking_flow.completed
assert no_horizontal_overflow
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.43 – Recovery after restart

Purpose. This annexure gives a focused, practical test design for recovery after restart in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

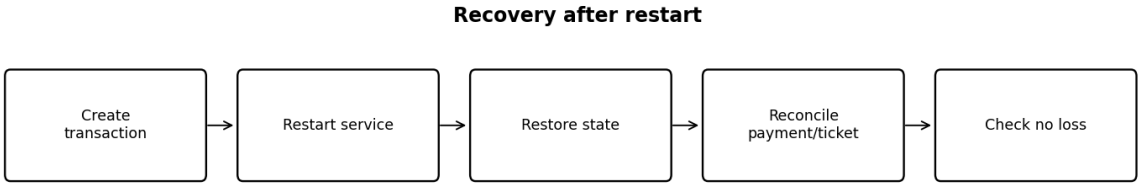


Figure A3.43. Recovery after restart – illustrative verification flow.

Representative automated check

```
assert recovered.payment_ref == original.payment_ref
assert recovered.no_duplicate_ticket
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.44 – Regression smoke suite

Purpose. This annexure gives a focused, practical test design for regression smoke suite in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

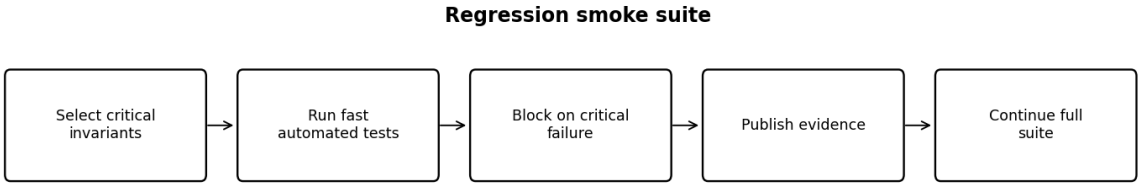


Figure A3.44. Regression smoke suite – illustrative verification flow.

Representative automated check

```
assert smoke_results["berth_uniqueness"] == "PASS"
assert smoke_results["payment_ticket_consistency"] == "PASS
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.45 – Impact-based selection

Purpose. This annexure gives a focused, practical test design for impact-based selection in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

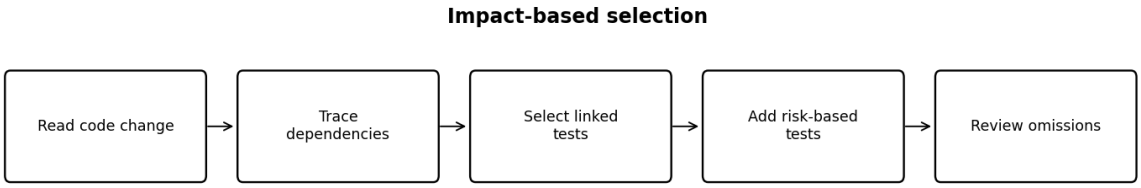


Figure A3.45. Impact-based selection – illustrative verification flow.

Representative automated check

```
selected = impact_analysis(changed_files, dependency_graph)
assert "refund_boundary_tests" in selected
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.46 – Prioritised execution

Purpose. This annexure gives a focused, practical test design for prioritised execution in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

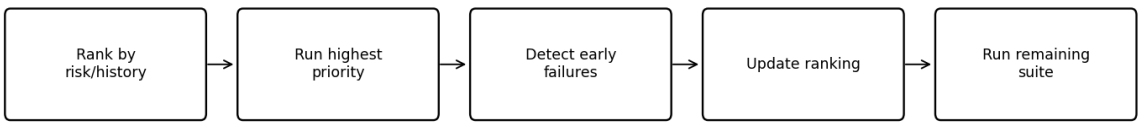


Figure A3.46. Prioritised execution – illustrative verification flow.

Representative automated check

```
ordered = sorted(tests, key=lambda t: t.risk_score, reverse=True)
assert ordered[0].risk_score >= ordered[-1].risk_score
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.47 – APFD calculation

Purpose. This annexure gives a focused, practical test design for apfd calculation in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

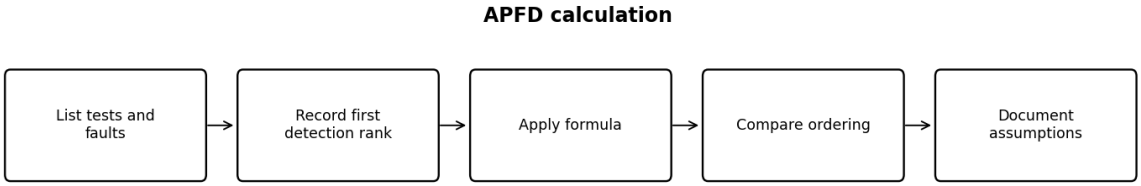


Figure A3.47. APFD calculation – illustrative verification flow.

Representative automated check

```
apfd = 1 - sum(first_detection_ranks)/(n*m) + 1/(2*n)
assert 0 <= apfd <= 1
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.48 – Obsolete test review

Purpose. This annexure gives a focused, practical test design for obsolete test review in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

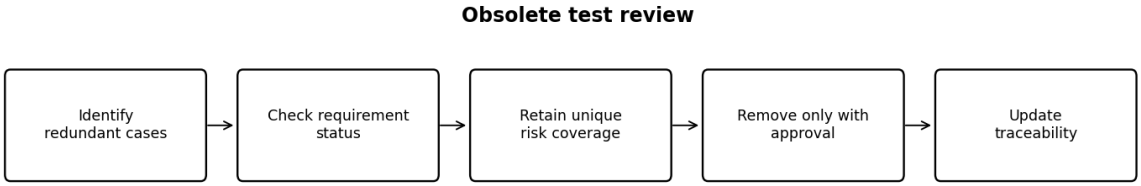


Figure A3.48. Obsolete test review – illustrative verification flow.

Representative automated check

```
assert obsolete_test.has_replacement_or_approved_removal
assert traceability.updated_at is not None
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.49 – Alpha test session

Purpose. This annexure gives a focused, practical test design for alpha test session in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

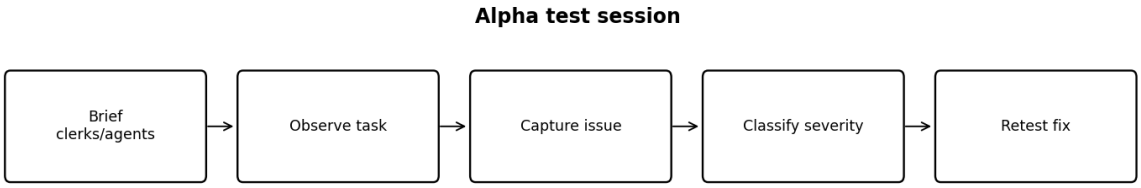


Figure A3.49. Alpha test session – illustrative verification flow.

Representative automated check

```
assert alpha_session.issue_log
assert each_issue.severity in ["Critical", "High", "Medium", "Low"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.50 – Beta feedback workflow

Purpose. This annexure gives a focused, practical test design for beta feedback workflow in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

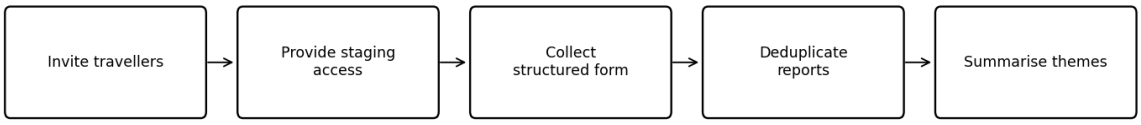


Figure A3.50. Beta feedback workflow – illustrative verification flow.

Representative automated check

```
assert beta_form.has_device_and_browser
assert beta_form.has_reproduction_steps
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.51 – Acceptance sign-off

Purpose. This annexure gives a focused, practical test design for acceptance sign-off in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

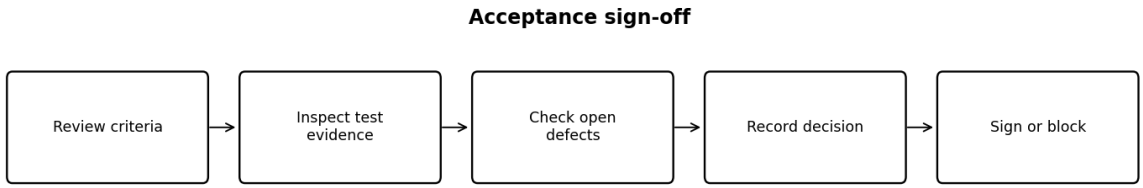


Figure A3.51. Acceptance sign-off – illustrative verification flow.

Representative automated check

```
assert acceptance_criteria.all_mandatory_pass
assert commercial_officer.decision in ["ACCEPT", "REJECT", "CONDITIONAL"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.52 – Release decision

Purpose. This annexure gives a focused, practical test design for release decision in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

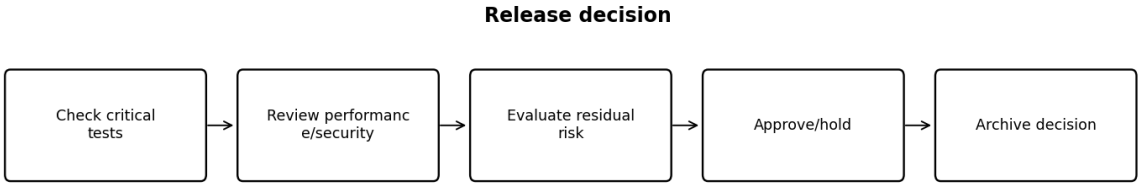


Figure A3.52. Release decision – illustrative verification flow.

Representative automated check

```
assert critical_test_evidence.complete
assert release_decision in ["APPROVE", "HOLD"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.53 – Balanced test pyramid

Purpose. This annexure gives a focused, practical test design for balanced test pyramid in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

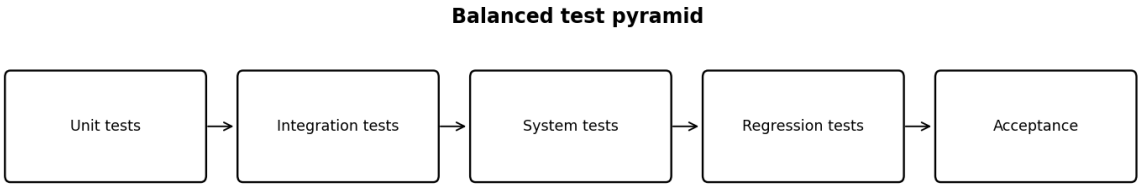


Figure A3.53. Balanced test pyramid – illustrative verification flow.

Representative automated check

```
assert unit_exit and integration_exit and system_exit
assert acceptance_signoff_recorded
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.54 – AI-generated test review

Purpose. This annexure gives a focused, practical test design for ai-generated test review in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

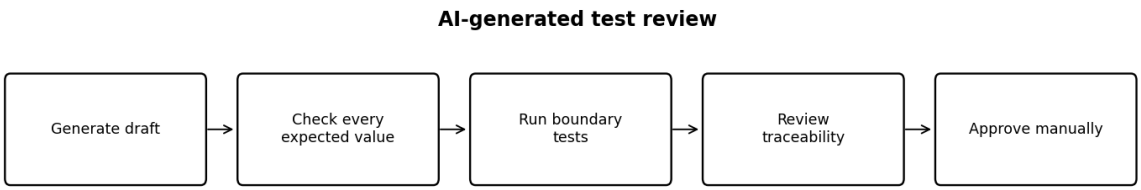


Figure A3.54. AI-generated test review – illustrative verification flow.

Representative automated check

```
assert generated_case.expected_value_reviewed_by_human
assert generated_case.requirement_id !=
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.55 – CI pipeline for fortnightly release

Purpose. This annexure gives a focused, practical test design for ci pipeline for fortnightly release in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

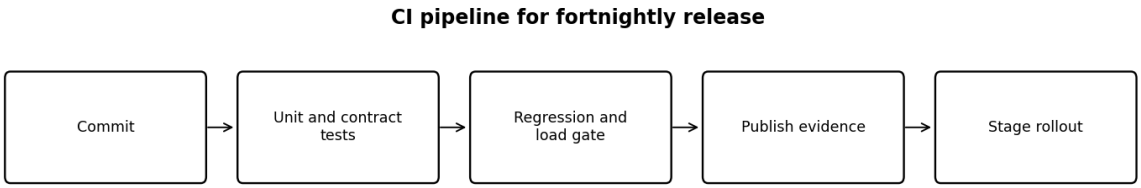


Figure A3.55. CI pipeline for fortnightly release – illustrative verification flow.

Representative automated check

```
assert ci_pipeline.unit_pass
assert ci_pipeline.contract_pass
assert ci_pipeline.regression_pass
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.

Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.56 – Consumer-driven contracts

Purpose. This annexure gives a focused, practical test design for consumer-driven contracts in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

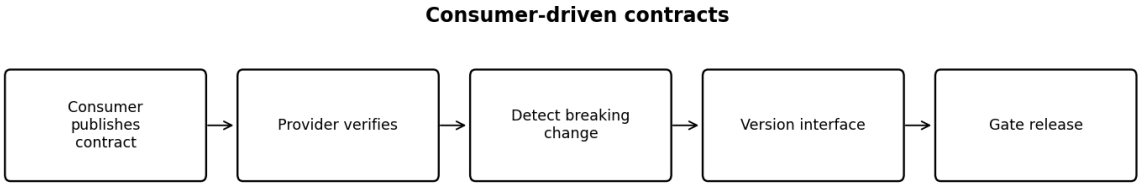


Figure A3.56. Consumer-driven contracts – illustrative verification flow.

Representative automated check

```
assert provider.verify(consumer_contract)
assert breaking_change_detected_before_release
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.57 – Canary rollout

Purpose. This annexure gives a focused, practical test design for canary rollout in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

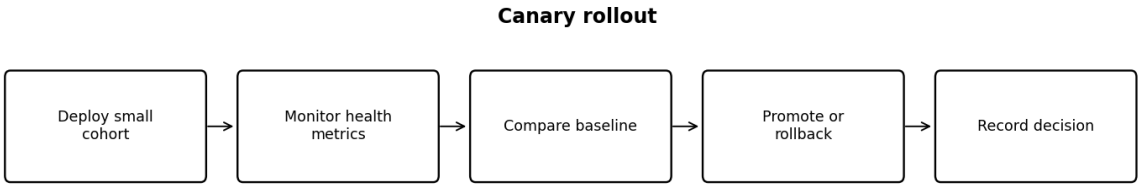


Figure A3.57. Canary rollout – illustrative verification flow.

Representative automated check

```
if canary_error_rate > threshold:
    rollback()
else:
    promote_to_next_cohort()
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.58 – 15,000-user load capacity

Purpose. This annexure gives a focused, practical test design for 15,000-user load capacity in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

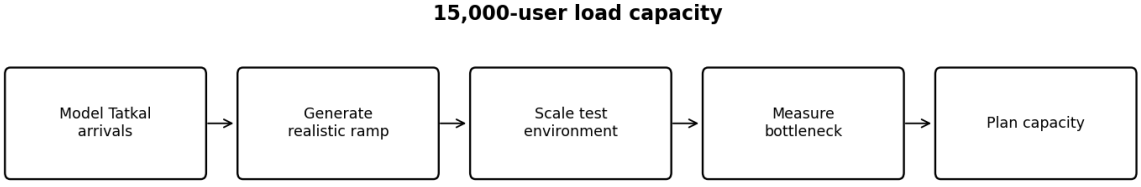


Figure A3.58. 15,000-user load capacity – illustrative verification flow.

Representative automated check

```
assert load_profile.peak_users == 15000
assert load_report.has_p90_and_error_rate
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.59 – Two-tester capacity

Purpose. This annexure gives a focused, practical test design for two-tester capacity in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

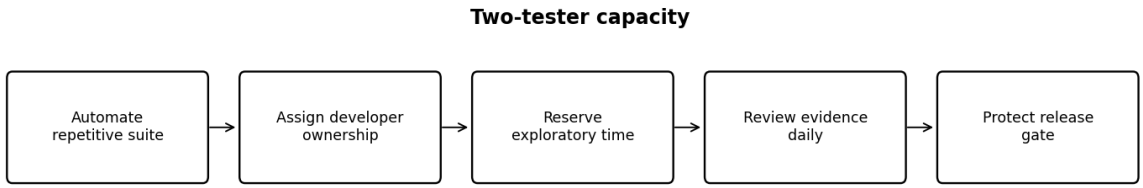


Figure A3.59. Two-tester capacity – illustrative verification flow.

Representative automated check

```
assert automated_suite.runtime_minutes <= budget
assert tester_review_time_is_reserved
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.60 – Audit evidence traceability

Purpose. This annexure gives a focused, practical test design for audit evidence traceability in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

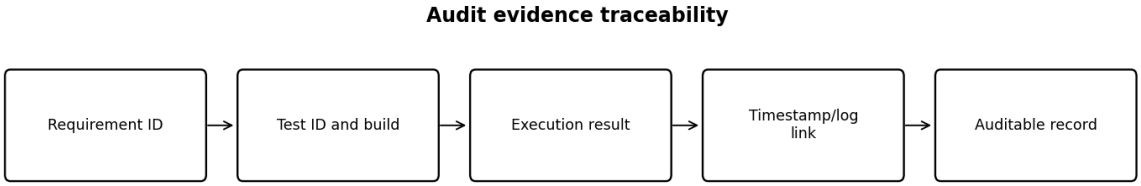


Figure A3.60. Audit evidence traceability – illustrative verification flow.

Representative automated check

```
assert evidence.requirement_id and evidence.test_id and evidence.build_id
assert evidence.status in ["PASS", "FAIL", "BLOCKED", "NOT RUN"]
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

ANNEXURE A3.61 – Final readiness review

Purpose. This annexure gives a focused, practical test design for final readiness review in the RailEase railway reservation system. It connects a stated risk to test inputs, expected behaviour, and the evidence that a reviewer should be able to reproduce.

Process. Use controlled test data and an isolated staging or prototype environment. Record the requirement, build identifier, test case, expected result, observed result, status, execution timestamp and any defect reference. The flow below is illustrative and must not be treated as proof of live execution.

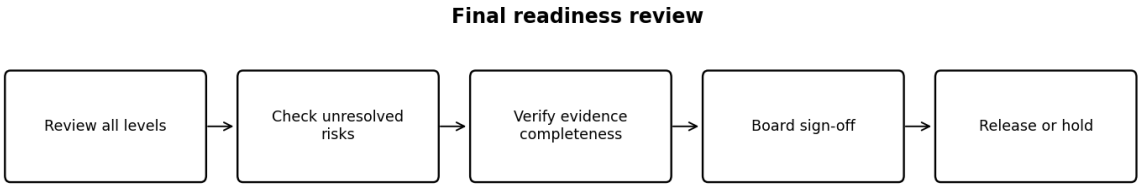


Figure A3.61. Final readiness review – illustrative verification flow.

Representative automated check

```
assert critical_test_evidence_complete
assert unresolved_critical_defects == 0
```

Test design and evidence

Test dimension	How to apply it	Evidence / expected outcome
Positive case	Use a valid value or permitted state transition.	Correct business outcome and recorded status.
Boundary / negative case	Exercise a policy boundary, invalid value, retry or failure path as relevant.	Safe rejection or specified alternate path; no inconsistent state.
Repeatability	Reset test data and repeat the case on the same build.	Same outcome under the same conditions; failures are reproducible.

Interpretation. A passing isolated example is useful but does not prove the entire system is correct. Combine the check with related tests at the appropriate level, particularly where concurrency, gateway behaviour, timeouts or persistent data are involved.

Reviewer checklist: requirement traceable; test data controlled; expected result independently checked; actual result recorded honestly; code and environment version captured; regression impact considered; unresolved risk documented.

CONSOLIDATED DESIGN DECISION TABLE

Decision	Justification (evidence to collect)	What if not adopted?
Plan-driven unit testing with BDD examples	Unit plan covers all business rules; boundary tests and seeded faults are traceable.	Ad hoc gaps may leave tax, age and per-passenger faults undetected.
Parameterized boundary tests plus branch coverage	Readable cases can be expanded through rows while branch reports reveal missing paths.	Either maintenance becomes repetitive or code paths remain untested.
Mocks plus gateway virtualisation	Mocks verify interaction; virtual service adds network delay, timeout and duplicate callback realism.	Mock-only confidence may not generalise to the real gateway.
Thread-based incremental integration	Early end-to-end journeys and stepwise fault isolation fit the five-week window.	Big-bang integration can defer fault discovery and lengthen diagnosis.
Hybrid interface/scenario/build plan	Covers every edge and the key booking/refund journeys with build gates.	Schedule may progress while a critical interface remains untested.
Risk-weighted system test mix	Functional integrity, load, recovery, security and compatibility address different failures.	A single focus area misses other known trial-week risks.
Prioritised regression with impact analysis	Fast critical suite plus linked tests balances time and risk; periodic full run checks omissions.	Retest-all consumes time; narrow selection can miss regressions.
Alpha → beta → formal acceptance	Find usability issues early, gather diverse feedback, then obtain accountable sign-off.	Acceptance becomes subjective or beta defects reach go-live.
Balanced pyramid for original scope	Defects are found near their source while system and user tests remain meaningful.	Top-heavy detection costs more; bottom-heavy testing misses real-world conditions.
Redesign for fortnightly releases	CI automation, contracts, load gates, canary and traceable evidence support two testers and 15,000-user peaks.	The old five-week plan cannot reliably meet the changed release and evidence constraints.

COMPARISON ACROSS OTHER SYSTEMS

System	Relative to RailEase	Testing emphasis

Airline seat allocation	Similar scarce-resource concurrency and high transaction value; often complex multi-leg itineraries.	Reservation invariants, concurrent seat allocation, payment reconciliation, schedule changes, distributed consistency and high-volume performance.
ATM network switch	Higher emphasis on security, availability, transaction integrity and recovery; strict timeout/reversal handling.	Protocol and contract testing, idempotency, reversal/reconciliation, penetration testing, failover and very high reliability.
School attendance app	Usually lower transaction criticality and lower peak load, but privacy and correctness remain important.	Role-based access, attendance edits/audit, mobile usability, reporting accuracy, privacy and practical acceptance by staff.

FINAL RECOMMENDATION

For the original RailEase scope, adopt a balanced pyramid with plan-driven unit testing, BDD examples for business rules, parameterised boundary suites, mocks supplemented by gateway virtualisation, incremental thread-based integration, a hybrid integration plan, repeated performance testing, prioritised regression and staged acceptance.

For the changed scope, REDESIGN the process around a CI pipeline and contract testing, with automated critical-test evidence linked to each build. Keep human acceptance for commercial rules and user workflows. Run a repeatable load profile that grows to 15,000 users in a properly sized environment, and use staged rollout with clear rollback thresholds.

The decision should be revisited if the organisation cannot provide stable test environments, reliable observability, automated evidence retention, or ownership for contract and load tests. In that situation, release frequency or scope must be reduced until the required assurance can be demonstrated.

EXECUTION EVIDENCE TEMPLATE

Field	Record for every executed test
Requirement / risk ID	Unique requirement or risk reference
Test case / level	Unit, integration, system, regression, alpha, beta or acceptance
Build / commit	Exact version under test
Environment / dataset	Staging/prototype, configuration and data version
Inputs and expected result	Values, state, expected business outcome
Observed result / status	Actual output and PASS / FAIL / BLOCKED / NOT RUN
Timing / metrics	Start-end time, response time, p90, error rate, coverage or effort as applicable
Defect / evidence links	Issue ID, log, screenshot, report or CI artifact
Reviewer / date	Person checking correctness and date of review